Xlib

Nathan Warner

Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

1. Introduction

- **The X Window System (X11)**: The X Window System, often called X11, is the general windowing system design and protocol. It defines things like:
  - how graphical programs communicate with a display server,
  - how windows are created, moved, resized, and drawn into,
  - how keyboard and mouse input is delivered,
  - how clients and the display server exchange messages.

  There is a distinction between a specification/standard and a concrete program that implements it.
- **X.Org display server**: The X.Org display server, usually called Xorg, is an actual open-source program that implements the X11 server side of that system. It is the software process that runs on your machine and speaks the X11 protocol to graphical applications
- **X clients**: The X.Org display server, usually called Xorg, is an actual open-source program that implements the X11 server side of that system. It is the software process that runs on your machine and speaks the X11 protocol to graphical applications
- **Windowing system**: A windowing system is the overall graphical environment model that lets programs use windows, input devices, and the screen in a coordinated way.
- **Display server**: A display server is the actual program that sits between graphical applications and the hardware/display system.
  - **Windowing system:** X Window System / X11
  - **Display server:** X.Org Server

  A display server is responsible for managing access to the screen, keyboard, mouse, touchpad, and sometimes GPU/display hardware. Graphical applications usually do not draw directly to the physical screen. Instead, they communicate with the display server.

  A windowing system is broader. It is the architecture that defines how graphical applications, windows, input, drawing, and the display server/compositor interact.

  It includes concepts such as:
  - windows;
  - graphical clients;
  - input events;
  - drawing surfaces;
  - focus;
  - stacking order;
  - protocols for communication between apps and the graphical environment.
- **Intro**: The X Window System is a network-transparent window system that was designed at MIT. X display servers run on computers with either monochrome or color bitmap display hardware. The server distributes user input to and accepts output requests from various client programs located either on the same machine or elsewhere in the network. Xlib is a C subroutine library that application programs (clients) use to interface with the window system by means of a stream connection. Although a client usually runs on the same machine as the X server it is talking to, this need not be the case.

Note: Monochrome means **one color**. It describes any visual, design, or medium that is constructed using only a single base hue along with its various tints, tones, and shades. While black-and-white is the most common example, a monochrome palette can be built entirely around any color

- **Network-transparent**: In general, network-transparent means that a system lets you use some resource over a network as if it were local, while hiding most of the networking details from the user or program.
- **Color bitmap**: A color bitmap is an image represented as a rectangular grid of pixels, where each pixel stores color information. A color bitmap uses more bits per pixel so that each pixel can represent many possible colors. In a common 24-bit RGB color bitmap, each pixel stores three color components:

  $$(R, G, B).$$

  Each component uses 8 bits, so each ranges from 0 to 255.
- **Screens and Display**: The X Window System supports one or more screens containing overlapping windows or subwindows. A screen is a physical monitor and hardware that can be color, grayscale, or monochrome. There can be multiple screens for each display or workstation. A single X server can provide display services for any number of screens. A set of screens for a single user with one keyboard and one pointer (usually a mouse) is called a **display**.
- **Window hierarchy**: All the windows in an X server are arranged in strict hierarchies. At the top of each hierarchy is a root window, which covers each of the display screens. Each root window is partially or completely covered by child windows. All windows, except for root windows, have parents. There is usually at least one window for each application program. Child windows may in turn have their own children. In this way, an application program can create an arbitrarily deep tree on each screen. X provides graphics, text, and raster operations for windows.

  A child window can be larger than its parent. That is, part or all of the child window can extend beyond the boundaries of the parent, but all output to a window is clipped by its parent.

  That means X will only actually display the part of the child window that lies inside the parent window. Anything outside the parent's visible rectangle is not drawn on the screen.

  Children of a window have overlapping locations, one of the children is considered to be on top of or raised over the others, thus obscuring them. Output to areas covered by other windows is suppressed by the window system unless the window has backing store.

  Child windows obscure their parents, and graphic operations in the parent window usually are clipped by the children
- **Border**: A window has a border zero or more pixels in width, which can be any pattern (pixmap) or solid color you like. A window usually but not always has a background pattern, which will be repainted by the window system when uncovered.
- **Coordinates**: Each window and pixmap has its own coordinate system. The coordinate system has the X axis horizontal and the Y axis vertical with the origin [0, 0] at the upper-left corner. Coordinates are integral, in terms of pixels, and coincide with pixel centers. For a window, the origin is inside the border at the inside, upper-left corner.

- **Expose event**: X does not guarantee to preserve the contents of windows. When part or all of a window is hidden and then brought back onto the screen, its contents may be lost. The server then sends the client program an Expose event to notify it that part or all of the window needs to be repainted. Programs must be prepared to regenerate the contents of windows on demand.
- **Drawables**: X also provides off-screen storage of graphics objects, called pixmaps. Single plane (depth 1) pixmaps are sometimes referred to as bitmaps. Pixmaps can be used in most graphics functions interchangeably with windows and are used in various graphics operations to define patterns or tiles. Windows and pixmaps together are referred to as drawables.
- **Xlib function requests**: Most of the functions in Xlib just add requests to an output buffer. These requests later execute asynchronously on the X server. Functions that return values of information stored in the server do not return (that is, they block) until an explicit reply is received or an error occurs. You can provide an error handler, which will be called when the error is reported.
- **XSync**: If a client does not want a request to execute asynchronously, it can follow the request with a call to XSync, which blocks until all previously buffered asynchronous events have been sent and acted on. As an important side effect, the output buffer in Xlib is always flushed by a call to any function that returns a value from the server or waits for input.
- **Resource ID**: Many Xlib functions will return an integer resource ID, which allows you to refer to objects stored on the X server. These can be of type **Window, Font, Pixmap, Colormap, Cursor, and GContext**, as defined in the file **<X11/X.h>**. These resources are created by requests and are destroyed (or freed) by requests or when connections are closed. Most of these resources are potentially sharable between applications, and in fact, windows are manipulated explicitly by window manager programs. Fonts and cursors are shared automatically across multiple screens. Fonts are loaded and unloaded as needed and are shared by multiple clients. Fonts are often cached in the server. Xlib provides no support for sharing graphics contexts between applications
- **Client programs and events**: Client programs are informed of events. Events may either be side effects of a request (for example, restacking windows generates Expose ev ents) or completely asynchronous (for example, from the keyboard). A client program asks to be informed of events. Because other applications can send events to your application, programs must be prepared to handle (or ignore) events of all types.

  Input events (for example, a key pressed or the pointer moved) arrive asynchronously from the server and are queued until they are requested by an explicit call (for example, **XNextEvent** or **XWindowEvent**). In addition, some library functions (for example, **XRaiseWindow**) generate Expose and **ConfigureRequest** events. These events also arrive asynchronously, but the client may wish to explicitly wait for them by calling **XSync** after calling a function that can cause the server to generate events.
- **Errors**: Some functions return **Status**, an integer error indication. If the function fails, it returns a zero. If the function returns a status of zero, it has not updated the return arguments. Because C does not provide multiple return values, many functions must return their results by writing into clientpassed storage. By default, errors are handled either by a standard library function or by one that you provide. Functions that return pointers to strings return NULL pointers if the string does not exist.

  The X server reports **protocol errors** at the time that it detects them. If more than one error could be generated for a given request, the server can report any of them.

Because Xlib usually does not transmit requests to the server immediately (that is, it buffers them), errors can be reported much later than they actually occur. For debugging purposes, howev er, Xlib provides a mechanism for forcing synchronous behavior. When synchronization is enabled, errors are reported as they are generated.

When Xlib detects an error, it calls an error handler, which your program can provide. If you do not provide an error handler, the error is printed, and your program terminates.

- **Standard Header Files**: The following include files are part of the Xlib standard:
  - **<X11/Xlib.h>**: This is the main header file for Xlib. The majority of all Xlib symbols are declared by including this file. This file also contains the preprocessor symbol XlibSpecificationRelease. This symbol is defined to have the 6 in this release of the standard. (Release 5 of Xlib was the first release to have this symbol.)
  - **<X11/X.h>**: This file declares types and constants for the X protocol that are to be used by applications. It is included automatically from <X11/Xlib.h>, so application code should never need to reference this file directly.
  - **<X11/Xcms.h>**: This file contains symbols for much of the color management facilities. All functions, types, and symbols with the prefix ''Xcms'', plus the Color Conversion Contexts macros, are declared in this file. <X11/Xlib.h> must be included before including this file.
  - **<X11/Xutil.h>**: This file declares various functions, types, and symbols used for inter-client communication and application utility functions, which are described in chapters 14 and 16. <X11/Xlib.h> must be included before including this file.
  - **<X11/Xresource.h>**: This file declares all functions, types, and symbols for the resource manager facilities. **<X11/Xlib.h>** must be included before including this file.
  - **<X11/Xatom.h>**: This file declares all predefined atoms, which are symbols with the prefix ''XA_''.
  - **<X11/cursorfont.h>**: This file declares the cursor symbols for the standard cursor font. All cursor symbols have the prefix ''XC_''.
  - **<X11/keysymdef.h>**: This file declares all standard KeySym values, which are symbols with the prefix ''XK_''. The KeySyms are arranged in groups, and a preprocessor symbol controls inclusion of each group. The preprocessor symbol must be defined prior to inclusion of the file to obtain the associated values. The preprocessor symbols are
    - XK_MISCELLANY
    - XK_XKB_KEYS
    - XK_3270
    - XK_LATIN1
    - XK_LATIN2
    - XK_LATIN3
    - XK_LATIN4
    - XK_KATAKANA
    - XK_ARABIC
    - XK_CYRILLIC
    - XK_GREEK

    - XK_TECHNICAL
    - XK_SPECIAL
    - XK_PUBLISHING
    - XK_APL
    - XK_HEBREW
    - XK_THAI
    - XK_KOREAN
  - **<X11/keysym.h>**: This file defines the preprocessor symbols listed above and then includes **<X11/keysymdef.h>**.
  - **<X11/Xlibint.h>**: This file declares all the functions, types, and symbols used for extensions. This file automatically includes **<X11/Xlib.h>**.
  - **<X11/Xproto.h>**: This file declares types and symbols for the basic X protocol, for use in implementing extensions. It is included automatically from **<X11/Xlibint.h>**, so application and extension code should never need to reference this file directly.
  - **<X11/Xprotostr.h>**: This file declares types and symbols for the basic X protocol, for use in implementing extensions. It is included automatically from **<X11/Xproto.h>**, so application and extension code should never need to reference this file directly.
  - **<X11/X10.h>**: This file declares all the functions, types, and symbols used for the X10 compatibility functions.

- **KeySyms**: A KeySym is X11's symbolic meaning for a keyboard key.

  A keyboard event gives you a **KeyCode**, which is closer to "which physical/logical key was pressed." A **KeySym** is closer to "what symbol does that key currently mean?" The Xlib docs define a KeyCode as a physical or logical key code and a KeySym as the encoding of the symbol on the keycap.

  A keycode by itself is not enough, because the same physical key can mean different things depending on keyboard layout and modifiers.

- **Atoms**: An Atom is an integer ID that stands for a string name known to the X server.

  X11 uses atoms heavily for **window properties, property types, selections**, and **ClientMessage** protocols. Xlib's documentation describes a window property as named, typed data; the property name has a unique identifier called an atom, and property types are also represented using atoms.

  Instead of constantly sending strings like:

```
"WM_DELETE_WINDOW"
"_NET_WM_NAME"
"UTF8_STRING"
"WM_PROTOCOLS"
```

  clients ask the server for an integer identifier:

```
Atom wm_delete = XInternAtom(display, "WM_DELETE_WINDOW",
    ↪ False);
Atom wm_protocols = XInternAtom(display, "WM_PROTOCOLS",
    ↪ False);
```

Then they use those Atom values in later protocol requests.

- **Extensions**: An **X extension** is an addition to the core X11 protocol. The core X protocol has a fixed set of requests, events, errors, and objects. Extensions allow the server and clients to support extra functionality without changing the original core protocol.

  Examples include

```
RANDR     resizing/rotating screens, monitor layout
XRender   improved 2D rendering/compositing primitives
XInput2   advanced input devices/events
XFixes    fixes and additions to older X behavior
Composite off-screen window compositing support
Shape     non-rectangular windows
GLX       OpenGL integration with X
```

  Before using an extension, a client usually checks whether the server supports it. Xlib provides functions such as:

```
XQueryExtension(display, "RANDR", &major, &first_event,
    ↪ &first_error);
```

- **Generic values and types**: The following symbols are defined by Xlib and used throughout the manual.
  - Xlib defines the type **Bool** and the Boolean values **True** and **False**.
  - **None** is the universal null resource ID or atom.
  - The type **XID** is used for generic resource IDs.
  - The type **XPointer** is defined to be char* and is used as a generic opaque pointer to data.
- **Naming and Argument Conventions within Xlib**: Xlib follows a number of conventions for the naming and syntax of the functions. Given that you remember what information the function requires, these conventions are intended to make the syntax of the functions more predictable.

  The major naming conventions are:

  - To differentiate the X symbols from the other symbols, the library uses mixed case for external symbols. It leaves lowercase for variables and all uppercase for user macros, as per existing convention.
  - All Xlib functions begin with a capital X.

- The beginnings of all function names and symbols are capitalized.
- All user-visible data structures begin with a capital X. More generally, anything that a user might dereference begins with a capital X.
- Macros and other symbols do not begin with a capital X. To distinguish them from all user symbols, each word in the macro is capitalized.
- All elements of or variables in a data structure are in lowercase. Compound words, where needed, are constructed with underscores (_).
- The display argument, where used, is always first in the argument list.
- All resource objects, where used, occur at the beginning of the argument list immediately after the display argument.
- When a graphics context is present together with another type of resource (most commonly,adrawable), the graphics context occurs in the argument list after the other resource. Drawables outrank all other resources.
- Source arguments always precede the destination arguments in the argument list.
- The x argument always precedes the y argument in the argument list.
- The width argument always precedes the height argument in the argument list.
- Where the x, y, width, and height arguments are used together, the x and y arguments
- always precede the width and height arguments.
- Where a mask is accompanied with a structure, the mask always precedes the pointer to the structure in the argument list.

- **Programming Considerations**: The major programming considerations are:
  - Coordinates and sizes in X are actually 16-bit quantities. This decision was made to minimize the bandwidth required for a given lev el of performance. Coordinates usually are declared as an int in the interface. Values larger than 16 bits are truncated silently. Sizes (width and height) are declared as unsigned quantities.
  - Keyboards are the greatest variable between different manufacturers' workstations. If you want your program to be portable, you should be particularly conservative here.
  - Many display systems have limited amounts of off-screen memory. If you can, you should minimize use of pixmaps and backing store.
  - The user should have control of his screen real estate. Therefore, you should write your applications to react to window management rather than presume control of the entire screen. What you do inside of your top-level window, however, is up to your application. For further information, see chapter 14 and the Inter-Client Communication Conventions

- **Character Sets and Encodings**: Some of the Xlib functions make reference to specific character sets and character encodings. The following are the most common:
  - **X Portable Character Set**: A basic set of 97 characters, which are assumed to exist in all locales supported by Xlib. This set contains the following characters:

a..z A..Z 0..9
!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
<space>, <tab>, <newline>

This set is the left/lower half of the graphic character set of ISO8859-1 plus space, tab, and newline. It is also the set of graphic characters in 7-bit ASCII plus the same three control characters. The actual encoding of these characters on the host is system dependent.

- **Host Portable Character Encoding**: The encoding of the X Portable Character Set on the host. The encoding itself is not defined by this standard, but the encoding must be the same in all locales supported by Xlib on the host. If a string is said to be in the Host Portable Character Encoding, then it only contains characters from the X Portable Character Set, in the host encoding.
  - **Latin-1**: The coded character set defined by the ISO8859-1 standard.
  - **Latin Portable Character Encoding**: The encoding of the X Portable Character Set using the Latin-1 codepoints plus ASCII control characters. If a string is said to be in the Latin Portable Character Encoding, then it only contains characters from the X Portable Character Set, not all of Latin-1.
  - **STRING Encoding**: Latin-1, plus tab and newline.
  - **POSIX Portable Filename Character Set**: The set of 65 characters, which can be used in naming files on a POSIX-compliant host, that are correctly processed in all locales. The set is:

$$a..z \quad A..Z \quad 0..9 \quad ._- .$$

- **Formatting Conventions**:
  - Global symbols are printed in this special font. These can be either function names, symbols defined in include files, or structure names. When declared and defined, function arguments are printed in italics. In the explanatory text that follows, they usually are printed in regular type.
  - Each function is introduced by a general discussion that distinguishes it from other functions. The function declaration itself follows, and each argument is specifically explained. Although ANSI C function prototype syntax is not used, Xlib header files normally declare functions using function prototypes in ANSI C environments. General discussion of the function, if any is required, follows the arguments. Where applicable, the last paragraph of the explanation lists the possible Xlib error codes that the function can generate.
  - To eliminate any ambiguity between those arguments that you pass and those that a function returns to you, the explanations for all arguments that you pass start with the word **specifies** or, in the case of multiple arguments, the word **specify**. The explanations for all arguments that are returned to you start with the word **returns** or, in the case of multiple arguments, the word **return**. The explanations for all arguments that you can pass and are returned start with the words **specifies** and **returns**.
  - Any pointer to a structure that is used to return a value is designated as such by the _return suffix as part of its name. All other pointers passed to these functions are used for reading only. A few arguments use pointers to structures that are used for both input and output and are indicated by using the _in_out suffix.

2. Display functions

- **Intro**: Before your program can use a display, you must establish a connection to the X server. Once you have established a connection, you then can use the Xlib macros and functions discussed in this chapter to return information about the display. We will discuss how to
  - Open (connect to) the display
  - Obtain information about the display, image formats, or screens
  - Generate a **NoOperation** protocol request
  - Free client-created data
  - Close (disconnect from) a display
  - Use X Server connection close operations
  - Use Xlib with threads
  - Use internal connections
- **Opening the Display**: To open a connection to the X server that controls a display, use **XOpenDisplay**

```
0  Display *XOpenDisplay(char* display_name)
```

  - **display_name** Specifies the hardware display name, which determines the display and communications domain to be used. On a POSIX-conformant system, if the $\text{display}_name is NULL, it defaults to the value of the DISPLAY environment variable.$

Note: The encoding and interpretation of the display name are implementation-dependent. Strings in the Host Portable Character Encoding are supported; support for other characters is implementation-dependent. On POSIX-conformant systems, the display name or DISPLAY environment variable can be a string in the format:

protocol/hostname:number.screen_number

  - **protocol**: Specifies a protocol family or an alias for a protocol family. Supported protocol families are implementation dependent. The protocol entry is optional. If protocol is not specified, the / separating protocol and hostname must also not be specified.
  - **hostname**: Specifies the name of the host machine on which the display is physically attached. You follow the hostname with either a single colon (:) or a double colon (::).
  - **number**: Specifies the number of the display server on that host machine. You may optionally follow this display number with a period (.). A single CPU can have more than one display. Multiple displays are usually numbered starting with zero.
  - **screen_number**: Specifies the screen to be used on that server. Multiple screens can be controlled by a single X server. The screen_number sets an internal variable that can be accessed by using the **DefaultScreen** macro or the **XDefaultScreen** function if you are using languages other than C

For example, the following would specify screen 1 of display 0 on the machine named ‘‘dualheaded’’:

The **XOpenDisplay** function returns a **Display** structure that serves as the connection to the X server and that contains all the information about that X server. **XOpenDisplay** connects your application to the X server through TCP or DECnet communications protocols, or through some local inter-process communication protocol. If the protocol is specified as "tcp", "inet", or "inet6", or if no protocol is specified and the hostname is a host machine name and a single colon (:) separates the hostname and display number, **XOpenDisplay** connects using **TCP streams**.

If the protocol is specified as "inet", TCP over IPv4 is used. If the protocol is specified as "inet6", TCP over IPv6 is used. Otherwise, the implementation determines which IP version is used. If the hostname and protocol are both not specified, Xlib uses whatever it believes is the fastest transport. If the hostname is a host machine name and a double colon (::) separates the hostname and display number, **XOpenDisplay** connects using DECnet. A single X server can support any or all of these transport mechanisms simultaneously. A particular Xlib implementation can support many more of these transport mechanisms.

If successful, **XOpenDisplay** returns a pointer to a **Display** structure, which is defined in **<X11/Xlib.h>.** If **XOpenDisplay** does not succeed, it returns NULL. After a successful call to **XOpenDisplay,** all of the screens in the display can be used by the client. The screen number specified in the display_name argument is returned by the **DefaultScreen** macro (or the **XDefaultScreen** function). You can access elements of the **Display** and **Screen** structures only by using the information macros or functions.

- **Obtaining Information about the Display, Image Formats, or Screens**: The Xlib library provides a number of useful macros and corresponding functions that return data from the Display structure. The macros are used for C programming, and their corresponding function equivalents are for other language bindings.

  All other members of the Display structure (that is, those for which no macros are defined) are private to Xlib and must not be used. Applications must never directly modify or inspect these private members of the Display structure.

- **Display Macros**: Applications should not directly modify any part of the Display and Screen structures. The members should be considered read-only, although they may change as the result of other operations on the display.

3. Window functions

4. Window information functions

5. Pixmap and cursor functions

6. Color management functions

7. Graphics context functions

8. Graphics functions

9. Window and session manager functions

10. Events

11. Event handling functions

12. Input device functions